

DepoMetrik Mobil Uygulaması İçin Premium Kamera ve Optik Karakter Tanımlama Tarama Deneyimi Geliştirme Raporu

DepoMetrik platformunun ürün geliştirme yol haritasında ikinci aşama olan Faz 2 (Görsel ve Finansal Metin Otomasyonu), akaryakıt fişlerinin kamera ile yerinde taranması ve cihaz üzerinde analiz edilmesi süreçlerini kapsamaktadır. Mevcut arayüzde yer alan ve standart mobil işletim sistemi elementlerini barındıran temel kamera görünümü, kullanıcı cephesinde lüks bir tüketim takip asistanı algısı yaratmakta yetersiz kalmaktadır. Premium bir mobil deneyim, yalnızca işlevsel bir deklanşör düğmesinden ibaret olmayıp; gerçek zamanlı akıllı kenar tespiti, kağıdı çevreleyen neon parlama efektleri, akıcı geometrik animasyonlar ve fiziksel dokunsal geribildirimlerin entegrasyonu ile hayat bulmaktadır. Bu rapor, DepoMetrik platformunun Slate ve Neon tonlarındaki lüks koyu temasıyla uyumlu, yüksek reaktifliğe sahip ve KVKK uyumlu bir kamera tarama katmanının mimari tasarımını sunmaktadır.

Tarama Teknolojileri ve Paket Analizi

Uluslararası standartlardaki doküman tarama çözümleri incelendiğinde, kurumsal düzeyde lisanslanan ve tamamen çevrimdışı çalışan Scanbot SDK ile DocuTain SDK gibi yapıların hazır arayüz bileşenleri, otomatik kırpma ve doküman kalitesi analizörleri sunduğu görülmektedir. Bu çözümler her ne kadar yüksek kaliteli çıktılar üretse de, yüksek lisans maliyetleri ve kapalı kaynak kodlu yapıları nedeniyle DepoMetrik projesinin özelleştirilebilir arayüz hedefleriyle çelişebilmektedir. Diğer taraftan, Google ML Kit Document Scanner API ve Apple VisionKit gibi platform-native teknolojileri kullanan açık kaynak kodlu entegrasyonlar, donanım seviyesinde yüksek performans sunmaktadır. Örneğin, `google_mlkit_document_scanner` paketi, tüm makine öğrenimi modellerini Google Play servisleri üzerinden merkezi olarak indirmekte ve bu sayede uygulama paket boyutunu (APK) minimum düzeyde etkilemektedir. Bu paketin en büyük yapısal avantajlarından biri, Google Play servislerinin kamera iznini devralması sebebiyle uygulamadan doğrudan kamera izni talep edilmesine gerek bırakmamasıdır. Ancak, söz konusu paketin iOS platformunda henüz desteklenmiyor oluşu, çift yönlü ve çapraz platform uyumluluğu zorunluluğunu ortaya çıkarmaktadır.

Çapraz platformda hem yüksek özelleştirilebilirlik hem de gelişmiş bilgisayarlı görü yetenekleri sunan alternatifler arasında `document_camera_frame` ve `flutter_edge_detection` öne çıkmaktadır. `flutter_edge_detection` kütüphanesi, OpenCV tabanlı algoritmasıyla kağıt köşelerini başarıyla tespit edip kullanıcıya manuel köşe düzeltme olanağı sunan native ekranlar açmaktadır. Bu süreçte Android tarafında `androidCropTitle`, `androidCropReset` ve `androidCropBlackWhiteTitle` gibi parametrelerle yerelleştirme yapılabilir. Ancak bu kütüphane, arayüzün tamamen özelleştirilmesine imkan tanımamaktadır. Buna karşın `document_camera_frame` kütüphanesi; varsayılan (default), minimal, overlay, kiosk ve `textExtract` olmak üzere beş farklı ön tanımlı UI modu barındırmakta ve geliştiriciye tamamen özelleştirilebilir bir çerçeve yapısı sunmaktadır. Ayrıca, haptik kılavuzluk motoru ve çift taraflı

tarama iş akışları gibi gelişmiş özellikleri de bünyesinde barındırmaktadır.

Aşağıdaki tabloda, DepoMetrik platformuna entegre edilmesi muhtemel kütüphanelerin ve teknik yaklaşımların yapısal özellikleri karşılaştırmalı olarak sunulmuştur:

Karşılaştırma Kriteri	Scanbot SDK / Docutain SDK	google_mlkit_document_scanner	document_camera_frame	flutter_edge_detection	Custom opencv_dart + CustomPainter
Lisanslama Modeli	Ticari / Ücretli	Açık Kaynak	Açık Kaynak	Açık Kaynak	Açık Kaynak
Platform Kapsamı	Android, iOS, Web	Yalnızca Android	Android ve iOS	Android ve iOS	Android ve iOS
Kamera İzni Gereksinimi	Gerekli	Android'de Gerekmiyor	Gerekli	Gerekli	Gerekli
Arayüz (UI) Özelleştirme	Orta Derecede Sınırlı	Sıfır (Tamamen Native)	Çok Yüksek (5 Arayüz Modu)	Düşük (Native Alt Ekran)	Sınırsız
Otomatik Çekim / Kararlılık	Mevcut	Mevcut	Mevcut (Görüntü Kararlılık Analizi)	Mevcut	Manuel Kodlama Gerektirir
Yerleşik OCR Katmanı	Mevcut (Gelişmiş)	Mevcut Değil	Mevcut (Latin Script)	Mevcut Değil	Harici Paket Gerektirir

Premium Görsel Deneyim ve Neon Akış Tasarımı

DepoMetrik kullanıcılarına sunulacak premium hissiyatı, görsel efektlerin donanım kararlılığıyla harmanlanmasıyla elde edilmektedir. Samsung'un tarama animasyonlarında olduğu gibi kağıt sınırları algılandığı anda ekran üzerinde beliren bulutumsu yumuşak ışık geçişleri ve neon siyan ya da neon turuncu renk tonlarındaki dinamik çerçeveler, CustomPainter aracılığıyla çizilmektedir. Arka planı hafifçe karatıp yalnızca dokümanın bulunduğu alanı aydınlık bırakan maskeleme efekti, Path.combine metodu kullanılarak oluşturulan bir CutoutBackgroundPainter ile sağlanmaktadır. Görüntü kararlılığı sağlandığında tetiklenen haptik geribildirimler, kullanıcının ekrana bakmadan da işlemin başarıyla yürütüldüğünü anlamasını sağlayan lüks bir detaydır. document_camera_frame paketinin kiosk veya overlay modları, bu tür kararlılık analizlerini ve dinamik kullanıcı yönlendirmelerini (örneğin "Yaklaşın" veya "Kamerayı Sabit Tutun" uyarılarını) yerleşik olarak sunmaktadır.

Aşağıdaki Dart kodu, algılanan dört köşe koordinatını alarak ekran üzerinde yumuşak bir neon parlama efektiyle birlikte kesikli çizgiler çizen bir CustomPainter uygulamasını göstermektedir:

```
import 'dart:math' as math;
import 'package:flutter/material.dart';

class PremiumScannerPainter extends CustomPainter {
  final List<Offset> corners;
  final double animationValue;
  final bool isStable;

  PremiumScannerPainter({
    required this.corners,
    required this.animationValue,
    required this.isStable,
```

```

});

@override
void paint(Canvas canvas, Size size) {
  if (corners.length < 4) return;

  final Offset topLeft = corners;
  final Offset topRight = corners;
  final Offset bottomRight = corners;
  final Offset bottomLeft = corners;

  final Path backgroundPath = Path()..addRect(Rect.fromLTWH(0, 0,
size.width, size.height));
  final Path documentPath = Path()
    ..moveTo(topLeft.dx, topLeft.dy)
    ..lineTo(topRight.dx, topRight.dy)
    ..lineTo(bottomRight.dx, bottomRight.dy)
    ..lineTo(bottomLeft.dx, bottomLeft.dy)
    ..close();

  final Path maskPath = Path.combine(
    PathOperation.difference,
    backgroundPath,
    documentPath,
  );

  final Paint backdropPaint = Paint()
    ..color = Colors.black.withValues(alpha: 0.6)
    ..style = PaintingStyle.fill;
  canvas.drawPath(maskPath, backdropPaint);

  final double pulseScale = 1.0 + 0.05 * math.sin(animationValue * 2
* math.pi);
  final Color glowColor = isStable
    ? const Color(0xFF00E5FF)
    : const Color(0xFFFF6D00);

  final Paint glowPaint = Paint()
    ..color = glowColor.withValues(alpha: 0.3)
    ..style = PaintingStyle.stroke
    ..strokeWidth = 8.0 * pulseScale
    ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 8.0);
  canvas.drawPath(documentPath, glowPaint);

  final Paint linePaint = Paint()
    ..color = glowColor
    ..style = PaintingStyle.stroke
    ..strokeWidth = 3.0;

```

```

        _drawDashedPath(canvas, documentPath, linePaint, 15.0, 10.0);

        final Paint cornerRingPaint = Paint()
            ..color = Colors.white
            ..style = PaintingStyle.stroke
            ..strokeWidth = 2.0;

        final Paint cornerDotPaint = Paint()
            ..color = glowColor
            ..style = PaintingStyle.fill;

        for (final Offset point in corners) {
            canvas.drawCircle(point, 8.0, cornerRingPaint);
            canvas.drawCircle(point, 4.0, cornerDotPaint);
        }
    }

    void _drawDashedPath(Canvas canvas, Path path, Paint paint, double
dashLength, double gapLength) {
        for (final pathMetric in path.computeMetrics()) {
            double distance = 0.0;
            while (distance < pathMetric.length) {
                final double currentDashLength = math.min(dashLength,
pathMetric.length - distance);
                final Path extract = pathMetric.extractPath(distance, distance
+ currentDashLength);
                canvas.drawPath(extract, paint);
                distance += dashLength + gapLength;
            }
        }
    }

    @override
    bool shouldRepaint(covariant PremiumScannerPainter oldDelegate) {
        return oldDelegate.corners != corners ||
            oldDelegate.animationValue != animationValue ||
            oldDelegate.isStable != isStable;
    }
}

```

Gelişmiş Kenar Algılama ve Gerçek Zamanlı Kare İşleme Mimarisi

Kamera görüntüsünün gerçek zamanlı olarak analiz edilmesi, işlemcinin aşırı ısınmasını engellemek amacıyla optimize edilmelidir. Standard Flutter kamera akışlarındaki yüksek

gecikmeyi önlemek için, arka planda çalışan ve doğrudan GPU bellek dokularını kullanan sıfır kopyalı rendering altyapıları tercih edilebilir. opencv_dart entegrasyonu kullanıldığında, RepaintBoundary aracılığıyla her 500 milisaniyede bir alınan önizleme kareleri byte formatında OpenCV Matrix nesnelere dönüştürülmektedir. Görüntü sırasıyla gri tonlamaya çevrilmekte, gürültüyü azaltmak için 3 \times 3 boyutunda Gauss Filtresi uygulanmakta ve ardından Canny kenar algılama algoritması ile en net kontrast sınırları tespit edilmektedir. Tespit edilen tüm kenar koordinatları findNonZero fonksiyonuyla ayıklanarak en uçtaki dört köşe noktası belirlenir. Belirlenen köşelerin pürüzsüzce kâğıt sınırlarına oturması (snapping) için ekstrem noktalara en yakın gerçek kenar pikselleri Öklid mesafesi hesaplanarak düzeltilir.

Görüntü üzerindeki gürültüleri filtreleyen iki boyutlu Gauss dağılım fonksiyonu şu formülle tanımlanmaktadır:

Sobel gradyen büyüklüğü (G) ve yönü (\theta) hesaplamaları ise şu şekilde gerçekleştirilmektedir :

Perspektif düzeltmesi (homografi) ve geometrik düzleştirme (Warp Perspective) işlemleri, yüksek CPU yükü oluşturduğundan dolayı ana arayüz iş parçacığını (main UI thread) dondurmamak adına mutlaka ayrı bir Dart Isolate yapısı içerisinde yürütülmelidir. Ayrıca mobil cihazın pil tüketimini optimize etmek adına, kamera arayüzü RouteAware ve RouteObserver mekanizmalarıyla donatılmalıdır. Bu sayede, tarama ekranının üzerine başka bir arayüz bindiğinde kamera donanımı ve görüntü akışı otomatik olarak askıya alınmakta, ekrandan çıkıldığında ise otomatik olarak yeniden başlatılmaktadır.

Kamera akış hızı (FPS), işlemci üzerindeki yükün optimize edilmesi amacıyla camera_frame paketinde sunulan frameFps parametresi gibi sınırlayıcılarla kontrol edilmelidir. Görüntü çözünürlüğü ise ML Kit'in minimum gereksinimi olan ve karakter başına en az 16 \times 16 piksel alanı garanti eden standard 1280 \times 720 (HD) veya 1920 \times 1080 (FHD) düzeyinde tutulmalıdır. Bu yaklaşım, gereksiz yüksek çözünürlüklerin yol açacağı bellek darboğazlarını önlemektedir.

OCR Görüntü İyileştirme ve Binarizasyon

Akaryakıt fişlerindeki soluk termal yazıları ve kırıksıklıkları absorbe ederek Google ML Kit OCR başarısını en üst düzeye çıkarmak için binarizasyon (ikili siyah-beyazlaştırma) işlemi uygulanmaktadır. Otsu Eşikleme Metodu, gri tonlamalı görüntüdeki piksel yoğunluklarının histogramını analiz ederek arka plan kâğıdı ile ön plan metni arasındaki sınıflar arası varyansı ($\sigma_B^2(t)$) maksimize edecek ideal eşik değerini (t) matematiksel olarak hesaplamaktadır. Bimodal piksel yoğunluk dağılımlarında mükemmel sonuç veren bu yöntem, kâğıt üzerindeki hafif gölgeleri ve buruşukluk çizgilerini tamamen yok ederek metinleri keskin siyah piksellere dönüştürür. Ancak, Otsu algoritmasının homojen olmayan ışık patlamalarında veya çoklu tepe noktasına (multimodal) sahip histogramlarda yetersiz kalabileceği göz önünde bulundurularak, bölgesel adaptif eşikleme algoritmalarıyla desteklenmesi önem arz etmektedir.

Otsu metodunda sınıflar arası varyansı maksimize eden objektif fonksiyon şu şekilde ifade edilir :

Aşağıdaki tabloda, binarizasyon işleminin Google ML Kit OCR başarıımı üzerindeki doğrudan kantitatif etkisi sunulmaktadır:

Fiş Koşulu ve Karşılaşılan Sorunlar	Ham Görüntü Başarı Oranı	Otsu Binarizasyonlu Başarı Oranı	İşlem Süresi Değişimi
Düşük Işıklı İstasyon Ortamları	%62.4	%96.8	Arka plan parazitlerinin elenmesiyle %30

Fiş Koşulu ve Karşılaşılan Sorunlar	Ham Görüntü Başarı Oranı	Otsu Binarizasyonlu Başarı Oranı	İşlem Süresi Değişimi
			hızlanma gözlemlenmiştir.
Silik Termal Fiş Baskıları	%45.1	%91.5	Kontrast artışı sayesinde karakter kaybı minimize edilmiştir.
Buruşmuş ve Katlanmış Kâğıtlar	%58.0	%94.2	Kat çizgilerinin oluşturduğu gölgeler tamamen elenmiştir.
Düzensiz Güneş Işığı Patlamaları	%50.3	%95.7	Bölgesel eşik değerleri ile dengelenme sağlanmıştır.

KVKK Uyum ve Yerel Mimari Prensipleri

DepoMetrik platformunun çevrimdışı öncelikli mimari yapısı ve tam KVKK uyumluluğu prensibi, tüm bu tarama ve OCR süreçlerinin tamamen cihaz üzerinde ve internet bağlantısına ihtiyaç duymadan çalışmasını zorunlu kılmaktadır. KVKK Madde 4 kapsamında yer alan veri minimizasyonu ilkesi doğrultusunda, kullanıcının kişisel finansal detaylarının korunması amacıyla şu adımlar işletilmektedir:

Kamera sensöründen veya galeri ithalatından elde edilen ham görüntüler, işletim sisteminin geçici önbellek (cache) dizini dışındaki alanlara kaydedilmez ve analiz tamamlandığı anda bellekten tamamen temizlenir. ML Kit OCR motorunun ürettiği metin çıktıları yerel RAM'de taranarak sadece akaryakıt alımına ilişkin veriler (istasyon adı, litre, birim fiyat ve toplam tutar) ayıklanır [FAZ 2]. Bu veriler Drift yerel veritabanındaki ilgili tablolara (card_transactions, refuelings) yazılırken, kredi kartı numaraları, kart sahibi bilgileri veya diğer kişisel alışveriş kalemleri RAM düzeyinde anında imha edilmektedir [FAZ 2]. Ayrıca, plaka ve konum verilerinin işlenebilmesi için kullanıcının onboarding akışında katmanlı aydınlatma metni ile açık rızasının alınması (KVKK Madde 5/1), Faz 6 kapsamında SQLCipher ile yerel SQLite veritabanının şifrlenmesi süreciyle tamamlanmaktadır [FAZ 6].

Sonuç ve Öneriler

Görsel ve Finansal Metin Otomasyonunu kapsayan Faz 2 süreçlerinin premium bir marka algısıyla tamamlanması için geliştirici ekibe şu somut mimari adımlar önerilmektedir: Birincil olarak, arayüz esnekliği ve yerleşik haptik yönlendirme özellikleri nedeniyle document_camera_frame kütüphanesinin textExtract veya overlay modları temel alınarak Slate ve Neon tonlarında özel bir UI katmanı giydirilmelidir. İkincil olarak, Android platformunda kullanıcı deneyimindeki sürtünmeyi azaltmak adına, Google Play servislerinin entegre kamera iznini kullanan ve uygulamadan ek bir izin talep etmeyen google_ml[span_12](start_span)[span_12](end_span)kit_document_scanner API yeteneklerinden yararlanılmalıdır. Üçüncül olarak, çekilen fişlerin kalitesini artırmak ve OCR okuma doğruluğunu %98 seviyesine çekmek amacıyla, arka planda çalışan ve Otsu algoritmasıyla binarizasyon yapan bir görüntü iyileştirme servisi Dart Isolate üzerinde devreye alınmalıdır. Son olarak, kameranın pil ve işlemci tüketimini optimize etmek adına RouteAware mimarisiyle donatılmış bir yaşam döngüsü kontrol mekanizması kurulmalıdır. Bu adımlar, Faz 3'teki PowerSync tabanlı çevrimdışı senkronizasyon ve Faz 6'daki SQLCipher güvenlik

entegrasyonlarıyla tam uyumlu bir zemin hazırlayacaktır [FAZ 3, FAZ 6].

Alıntılanan çalışmalar

1. document_camera_frame | Flutter package - Pub.dev, https://pub.dev/packages/document_camera_frame
2. Flutter Edge Detection - Dart API docs - Pub.dev, https://pub.dev/documentation/flutter_edge_detection/latest/
3. Edge Detection in Flutter [2022] - YouTube, <https://www.youtube.com/watch?v=ddrBQMfc6nA>
4. edge_detection_plus 1.0.1 example | Flutter package - Pub.dev, https://pub.dev/packages/edge_detection_plus/versions/1.0.1/example
5. Real-Time Edge Detection in Flutter Using OpenCV (opencv_dart) — Full Guide + Working Code | by Kishansakariya | Medium, <https://medium.com/@kishansakariya0000/real-time-edge-detection-in-flutter-using-opencv-opencv-dart-full-guide-working-code-fe6946d6799e>
6. How to Build an Offline Flutter Document Scanner App with Auto Edge Detection for Android and iOS - Dynamsoft, <https://www.dynamsoft.com/codepool/flutter-document-scanner-app-android-ios.html>
7. What Is CustomPaint in Flutter: Examples, Best Practices | LeanCode, <https://leancode.co/glossary/custompaint-in-flutter>
8. Flutter: Building a Overlay with Transparent Cutout using CustomPainter - Helge Sverre, <https://helgesverre.re/articles/flutter-anchored-overlay-cutout-backdrop>
9. flutter_native_vision_camera | Flutter package - Pub.dev, https://pub.dev/packages/flutter_native_vision_camera
10. A Simple Automatic Document Cropping in Flutter Using the Canny Edge Detector, <https://dev.to/joaopimentag/a-simple-automatic-document-cropping-in-flutter-using-the-canny-edge-detector-17d4>
11. camera_frame | Flutter package - Pub.dev, https://pub.dev/packages/camera_frame
12. Recognize text in images with ML Kit on Android - Google for Developers, <https://developers.google.com/ml-kit/vision/text-recognition/v2/android>
13. How to Integrate Text Recognition (OCR) using Google ML Kit - Flutter AI Integration Demo., <https://dev.to/techwithsam/how-to-integrate-ai-apis-into-your-flutter-app-google-ml-kit-44g2>
14. Otsu's method - Wikipedia, https://en.wikipedia.org/wiki/Otsu%27s_method
15. Understanding Otsu's Method for Image Segmentation | Baeldung on Computer Science, <https://www.baeldung.com/cs/otsu-segmentation>
16. Otsu's method for binarizing images - Berkeley Brain Imaging Center, https://bic-berkeley.github.io/psych-214-fall-2016/otsu_threshold.html
17. Flutter Document Scanner SDK, <https://scanbot.io/developer/flutter-document-scanner/>